Neural Network Compression

Verified results, negative findings, and a verification report for LeNet-5 / MNIST compression

---

Compressing LeNet-5 on MNIST with five techniques — magnitude pruning, INT8/low-bit quantization, unit clustering, L0-regularized structured sparsity, and quantization-aware training — with a bias toward **checking** results rather than reporting them. Every headline number in this report has been either repeated across seeds, cross-validated against an independent implementation, or materialized as a real artifact on disk.

Two of the seven headline results are **negative**. They are kept because they are the useful part: a compression method's advantage that vanishes under scrutiny is exactly the kind of finding a project like this should surface.

Headline Results

Result	Number	Evidence
Best compression within 1pp	147.82x at 98.49% (0.54pp loss)	L0 + INT4 QAT, 2 seeds
Best near-free config	81.51x at 98.93% (0.11pp loss)	L0 + INT4 QAT, 2 seeds
Best without QAT	26.20x at 98.97% (0.07pp loss)	L0 + INT8, 3 seeds
Highest sparsity at ~1% error	95% weights zero, 1.03% error	iterative prune + fine-tune, 3 seeds
Inference speedup from 10.4x fewer params	none (1.01x / 0.90x)	measured on materialized model
Deterministic annealing's edge over k-means	none under matched compute	paired t-test, n=5
Depth of the 90%-sparsity accuracy cliff	unstable, 69.55% ± 4.36pp	3 freshly trained baselines

Wording that the data actually supports:

- Compressed LeNet-5 **147x** (241 KB → 1.6 KB) at **0.54pp** accuracy loss (98.49% vs. 99.04% dense) by stacking learned L0 structured sparsity with INT4 quantization-aware training; storage accounting cross-validated against an independent implementation and against a materialized on-disk model.
- Reached **95% weight sparsity at 1.03% ± 0.06pp MNIST test error** (3 seeds) via iterative magnitude pruning with fine-tuning — recovering **+23pp** over the same pruning without retraining.
- Disproved a **+16pp** apparent advantage of deterministic-annealing clustering over k-means by matching per-layer optimization budgets; the gap fell to **−2.4pp** and was not statistically significant (paired t-test, n=5).
- Showed **10.4x fewer parameters produced no CPU inference speedup** (1.01x at batch 1, 0.90x at batch 64), demonstrating that FLOP reduction does not imply wall-clock gains at small model scale.

Note the distinction between **size** and **parameter count**: 147.82x is storage compression. The parameter count itself falls about 20x (the structural L0 part); INT4 shrinks bits-per-parameter, not the number of parameters. The two are not interchangeable.

What This Project Does NOT Show

Stated explicitly, because these are the claims a compression project is most often assumed to support:

- **No inference speedup.** Measured, and it isn't there (see above). Unstructured sparsity additionally yields no speedup on standard hardware without specialized sparse kernels, so the 95%-sparsity result is not a latency result either.
- **No real INT8/INT4 inference.** Quantization is simulated (quantize-then-dequantize). It measures accuracy impact correctly; it does not benchmark integer kernels.
- **No novel algorithm.** Deterministic annealing is Rose (1998), hard-concrete L0 gates are Louizos et al. (2017), iterative pruning is Han et al. (2015). This is careful reimplementation and evaluation.
- **Nothing multimodal, no graph models, no Markov chains.** MNIST images only.
- **One architecture, one dataset.** Every conclusion is about LeNet-5 on MNIST. The size accounting is literally hardcoded to this network's shape.

Results

Baseline: **99.04%** test accuracy, 241.0 KB dense fp32.

Pruning and quantization, single-shot (3 seeds)

Method	Setting	Accuracy	Size	Compression
Baseline	—	99.07% ± 0.02	241.0 KB	1.00x
Magnitude pruning	30% sparsity	99.07% ± 0.02	252.9 KB	0.95x
Magnitude pruning	50% sparsity	99.02% ± 0.09	180.9 KB	1.33x
Magnitude pruning	70% sparsity	98.36% ± 0.11	108.9 KB	2.21x
Magnitude pruning	90% sparsity	69.55% ± 4.36	36.9 KB	6.54x
INT8 quantization	per-tensor symmetric	99.07% ± 0.01	61.0 KB	3.95x

The seed here is the trained model, not the compression method — magnitude pruning and per-tensor INT8 are both deterministic given a fixed checkpoint, so each seed required training a fresh baseline. Every other experiment in this project is built on the seed-0 checkpoint, which scored 99.04%; accuracy losses quoted elsewhere are relative to that number, not to the 99.07% mean above.

Iterative pruning WITH fine-tuning (3 seeds)

A different protocol from the table above — retraining between pruning steps. Not comparable to the single-shot numbers and never to be quoted as one series.

Sparsity	Before f.t.	After f.t.	Error	Recovered	Size	Compr.
50%	99.02%	99.18% ± 0.06	0.82%	+0.16pp	180.9 KB	1.33x
70%	98.31%	99.18% ± 0.14	0.82%	+0.88pp	108.9 KB	2.21x
80%	97.72%	99.17% ± 0.04	0.83%	+1.45pp	72.9 KB	3.31x
90%	73.68%	99.08% ± 0.05	0.92%	+25.40pp	36.9 KB	6.54x
93%	83.70%	99.02% ± 0.08	0.98%	+15.31pp	26.1 KB	9.25x

Sparsity	Before f.t.	After f.t.	Error	Recovered	Size	Compr.
95%	75.81%	98.97% ± 0.06	1.03%	+23.16pp	18.9 KB	12.77x

Seed 0 alone reached 0.95% error at 95% sparsity; the 3-seed mean is 1.03%. Reporting the single best seed would have overstated the result by 0.08pp — small in absolute terms, but it is the difference between "beats 1% error" and "lands just above it".

L0 structured sparsity (5 seeds)

lambda	Accuracy	Compression
0.005	99.01% ± 0.09pp	4.26x ± 0.15
0.01	99.00% ± 0.03pp	6.96x ± 0.33
0.02	98.78% ± 0.06pp	10.93x ± 0.67

L0 + INT8 (3 seeds)

lambda	L0 alone	L0 + INT8	Size	Quantization cost
0.005	99.00% @ 4.25x	99.02% @ 16.69x	14.4 KB	−0.02pp
0.01	98.98% @ 6.70x	98.97% @ 26.20x	9.2 KB	+0.00pp
0.02	98.74% @ 11.06x	98.76% @ 42.91x	5.6 KB	−0.02pp

Pushing the stack to its limit (2 seeds, post-training quantization)

Pass mark is 1 percentage point of accuracy loss, i.e. ≥ 98.04%.

lambda	fp32	INT8	INT4	INT3	INT2
0.02	98.76% @ 10.63x	98.79% @ 41.27x	98.22% @ 79.55x	94.84% @ 103.47x	9.95% @ 148.27x
0.05	98.28% @ 20.46x	98.30% @ 78.18x	97.70% @ 147.82x	91.66% @ 189.83x	10.03% @ 266.05x
0.1	60.17% @ 214.44x	59.96% @ 670.40x	48.19% @ 1058x	30.37% @ 1230x	10.30% @ 1489x

Bold entries in the source data are the best passing configs (INT4 rows above, both lambda=0.02 and lambda=0.05). INT2 destroys the network outright (~10% = random guessing over 10 classes), and lambda=0.1 is past the structural cliff before any quantization is applied.

Quantization-aware training (2 seeds)

Config	PTQ accuracy	QAT accuracy	Recovered	Compression	Verdict
lambda=0.02, INT4 (n=2)	98.22% (−0.81pp)	98.93% (−0.11pp)	+0.71pp	81.51x	pass
lambda=0.05, INT4 (n=2)	97.70% (−1.34pp)	98.49% (−0.54pp)	+0.80pp	147.82x	pass

Config	PTQ accuracy	QAT accuracy	Recovered	Compression	Verdict
lambda=0.05, INT3 (n=4)	91.35% (−7.69pp)	97.93% (−1.11pp)	+6.58pp	184.58x	fail by 0.11pp

The lambda=0.02 ratio is 81.51x rather than the 79.55x its PTQ counterpart scored because INT4 rounding deleted whole units during fine-tuning, making the real model smaller than the pre-QAT report claimed; the run re-prices from the counts actually observed.

Unit clustering (5 seeds, deterministic annealing vs. k-means)

k	DA accuracy	k-means++ accuracy	Compression
4	14.21%	11.12%	18.09x
8	47.07%	32.73%	9.53x
16	85.22%	68.98%	5.14x
32	98.04%	93.97%	3.04x
64	98.84%	98.88%	1.67x

The DA column looks like a clear win. It isn't — see **Findings** below.

Findings

Positive / methodological

- **Sparse storage has a break-even point, and it's easy to miss.** At 30% sparsity the "compressed" model is larger than dense (0.95x). A stored nonzero costs 4 bytes (value) + 2 bytes (index) = 6, vs. 4 for a dense fp32 weight, so a naive sparse format only wins past 4/6 ≈ 33% sparsity. Reporting sparsity percentages without converting to a storage format hides this entirely.
- **The pruning cliff is real, but its depth is close to arbitrary.** Accuracy is essentially flat from 0% to 70% sparsity (99.07% → 98.36%, 0.71pp for more than 2x compression), then collapses by 90%. How far it collapses is the unstable part: 69.55% ± 4.36pp across 3 seeds, against spreads of 0.01–0.11pp for every other row in that table. Once pruning pushes a network past its cliff, a single-seed measurement of where it lands carries almost no information — an earlier draft of this project reported 63.66% from one pessimistic draw.
- **INT8 is the most stable measurement here** (±0.01pp across seeds) — a quiet argument for it as a default: not merely cheap in accuracy, but predictable in a way heavy pruning is not.
- **Fine-tuning is doing most of the work at high sparsity, and it should be labeled as such.** Single-shot pruning collapses to 69.55% at 90% sparsity; with retraining between steps the same sparsity reaches 99.08%. Averaged over 3 seeds the rescue is +25.40pp at 90% and +23.16pp at 95% — that is what the extra training bought, not what pruning achieved.
- **Structured L0 sparsity is the strongest single method, and it holds up.** At lambda=0.01 it gives 6.96x for 0.04pp, against INT8's 3.95x and magnitude pruning's 2.21x at comparable accuracy. Across 5 seeds the accuracy spread is ±0.03 to ±0.09pp while gaps between lambda settings are an order of magnitude larger — the curve's shape is a property of lambda, not the seed.
- **L0 and quantization stack almost perfectly, and the residual is the accounting being honest.** INT8 on top of L0 costs −0.02/+0.00/−0.02pp across lambda=0.005/0.01/0.02, measured paired on the same model before and after. INT8's multiplier decays 3.93x → 3.91x → 3.88x against 3.95x standalone, because biases stay fp32 under both schemes.

- **Quantization-aware training buys back most of INT4's loss.** Post-hoc INT4 at lambda=0.05 costs 1.34pp — enough to miss a 1pp budget at 147.82x. Fine-tuning through a straight-through-estimator quantizer recovers +0.80pp, landing at 0.54pp.
- **INT3's failure is a rounding problem, not a representational one — and it sits exactly on the budget line.** Under PTQ, INT3 looks hopeless (4.20pp at lambda=0.02, 7.69pp at lambda=0.05). QAT recovers +6.58pp of that 7.69pp gap, landing INT3 at 1.11pp and 184.58x. Across 4 seeds the individual QAT accuracies were 97.39, 97.89, 98.15 and 98.31% — 2 of 4 individually clear the 98.04% pass mark while the mean does not, so the honest verdict is “straddles the bar,” not a clean pass or fail.
- **QAT collapses seed variance, and most where PTQ is least stable.** Seed-to-seed spread before/after QAT: 0.81pp → 0.02pp (lambda=0.02/INT4), 0.14pp → 0.03pp (lambda=0.05/INT4), and 9.38pp → 0.92pp (lambda=0.05/INT3, n=4). This finding was wrong at n=2 — two seeds inverted the pattern purely by chance — and is kept as a documented example of how a two-seed spread estimate can invert a conclusion.
- **Coarse quantization can delete whole units by itself.** During INT4 QAT, a surviving unit whose weights are all small relative to its layer's per-tensor maximum rounded entirely to zero, changing the surviving-unit counts underneath the size accounting. The experiment detects this and re-prices from observed counts; a revived unit is treated as a hard error since that direction overstates compression.

Negative results

- **FLOP reduction did not produce a speedup.** The materialized narrow model — (6,16,120,84) → (6,11,14,10), 61,706 → 5,941 parameters, 10.39x fewer — ran at 1.01x at batch 1 and 0.90x (slower) at batch 64. At LeNet-5's scale, per-op framework overhead dominates arithmetic and narrow layers fall off SIMD-friendly alignment, so smaller matrices can run worse. Any “Nx smaller therefore Nx faster” claim needs a stopwatch, not a parameter count.
- **Deterministic annealing's advantage over k-means was a compute artifact.** DA appeared to beat k-means++ by +14.3pp at k=8 and +16.2pp at k=16. But DA runs far more inner-loop iterations per fit. Given a per-layer matched restart budget, the gap collapses to +3.8pp at k=8 and −2.4pp at k=16 — k-means++ wins there — and neither is significant on a paired t-test at n=5 (t=1.07, t=−0.56; |t|>2.776 needed). The sign isn't even consistent across k. DA just searches longer.
    - Matching compute has to be done per layer, not globally. A first version of the check summed DA's iterations across layers but averaged k-means' across layers, handing k-means++ 2–8x more than parity. Correct per-layer ratios span 135x (fc1) to 497x (conv1), because DA's iteration count is set by its temperature schedule and barely varies with layer size, while k-means' tracks how fast that layer's points settle.
    - Lower inertia does not mean better accuracy. At k=16, an over-generous global budget (n_init=1218) scored worse than the smaller per-layer-matched one (80.98% vs. 87.64%). Selecting restarts by inertia optimizes the wrong objective for preserving network function.
- **Unit clustering is the weakest method here.** Its best usable point (k=32: 98.04% at 3.04x) is dominated by INT8 (99.05% at 3.95x) on both axes, and buried by L0+INT8 (98.97% at 26.20x). Clustering whole units is too coarse for a network this small.

Verification

Numbers that could silently drift are pinned by checks designed to fail loudly rather than report a quietly wrong number:

- **Size accounting identity.** At 0% sparsity, `compressed_size_bytes_l0` returns 246,824 bytes — exactly the dense model — and priced as INT8 returns 3.953x, reproducing the 3.95x that `eval_baselines.py` computes through a separate code path.

- **Gate folding.** Hard-concrete gates are not binary at eval: $z = \text{clamp}(\text{sigmoid}(\text{log\_alpha}) \times 1.2 - 0.1, 0, 1)$ equals 1.0 only when $\text{log\_alpha} \geq 2.4$. Since $z \cdot (Wx+b) \equiv (zW)x + (zb)$, gates are folded into weights, which is what lets the accounting store nothing per unit. Every run asserts folding is accuracy-neutral to 1e-4.
- **Materialized model.** materialize_pruned.py builds the genuinely narrower network, transplants surviving weights through the cascade (each surviving conv2 channel owns 25 consecutive fc1 columns, not one), and asserts identical accuracy — drift was exactly 0.00e+00. A wrong index mapping breaks accuracy rather than quietly reporting a smaller number.
- **Structural sparsity under QAT.** Surviving-unit counts are re-derived after fine-tuning and compared against the pre-fine-tuning counts; a revived unit is treated as a hard error.
- **Selection on means, not rows.** Best-config selection aggregates per configuration across seeds. Picking the best individual seed row would report whichever run got lucky — at lambda=0.02/INT4 that would have turned a 79.55x @ 98.22% result into an 84.67x @ 98.63% one.

Scope and Limitations

- **Seed counts are low:** 5 for L0 and clustering, 3 for the baselines, L0+INT8, and iterative pruning, 2 for the stack sweeps. Non-significant results mean "not detectable at this n," not "no effect."
- **Quantization is simulated,** not real integer inference.
- **L0 size accounting is architecture-specific** — the cascade in compressed_size_bytes_l0 hardcodes this LeNet-5 (padding=2 conv1, two 2×2 pools, 5×5 map into fc1).
- **The iterative-pruning schedule was chosen after a shorter run missed the target.** The shorter run (2 epochs/step, no 93% step) reached 95% sparsity at 1.41% error; the reported schedule uses 4 epochs/step and an added 93% step. Both are retained in results/. The schedule was not re-tuned after seeing the 3-seed result.
- **Latency was measured on a CPU** under a Python/PyTorch runtime. A different runtime, or a model large enough to be compute-bound, could give a different answer.

Repository

Source: github.com/Stxtics03/Neural-Network-Comp. Structure:

Directory	Contents
models/	LeNet-5 (width-parameterizable, for materializing pruned nets)
data/	MNIST loading
compression/	Pruning, quantization (any bit width), clustering, L0 gates
eval/	Shared accuracy / size / compression-ratio metrics
experiments/	Training, evaluation, and verification entry points
results/	Output CSVs and checkpoints (checkpoints gitignored)